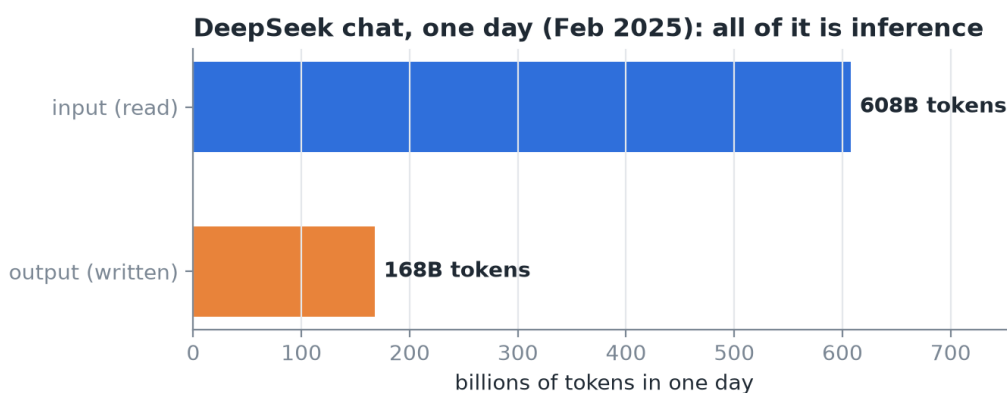


01 · What Is Inference & Inference Engineering?

Source: [fanout.sh / inference-eng / in-01](https://fanout.sh/inference-eng/in-01) · ~9 min · Next: *The Inference Stack*

TL;DR. Inference is running a trained model token by token, every time someone uses it, so across a fleet it usually costs more than training. Decode is limited by memory bandwidth (moving weights), not arithmetic. Latency, throughput and cost pull against each other, and batching is the first lever. The job is a loop: measure, find the bottleneck, change one thing, measure again.

1. Why inference matters: the DeepSeek bill



$\approx 1,800 \text{ GPUs} \times 24 \text{ h} \times \$2/\text{GPU-h} \approx \$87\text{k}$ for the day

DeepSeek chat service, 24 h, Feb 2025: ~1,800 GPUs on average, ~\$87k for the day at \$2/GPU-hour.

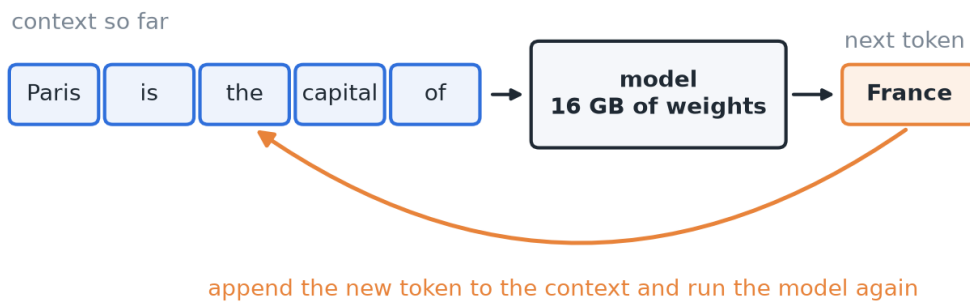
- One day of DeepSeek chat: **608B input tokens** read, **168B output tokens** written.
- Ran on **~1,800 GPUs** on average. At \$2/GPU-hour: $1,800 \times 24 \times \$2 \approx$ **\$87k/day**.
- The model was trained months earlier. Every one of those tokens was inference.

Key point: the gap between serving well and serving badly comes from a small number of engineering decisions. That gap is the job.

2. What inference actually is

- A trained model is **just a file of numbers**. Llama 3 8B has about 8B params at 16 bits (2 bytes each), so the file is **about 16 GB**.
- **Inference = running it**. Feed in text, predict the next token, append it, and repeat until done. This is **autoregressive** generation.

Autoregressive generation: one token per model run



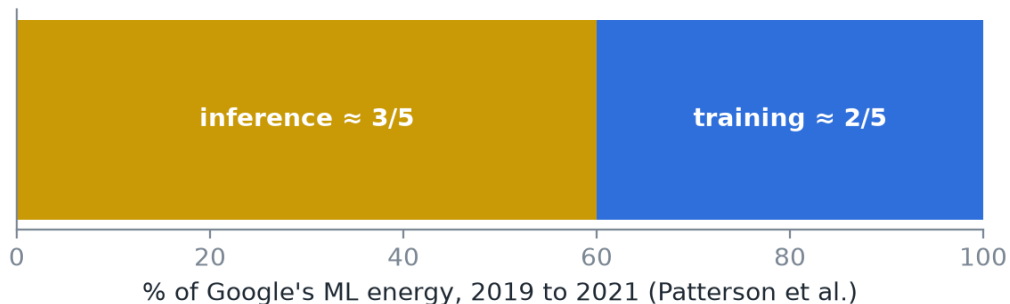
The autoregressive loop: each generated token is fed back in as input.

Training vs inference

	Training	Inference
How often	Once, or a few times	Every request, every chat, every agent step
Cost shape	Huge one-off	Ongoing, scales with usage

- Google fleet-wide, 2019 to 2021: **~3/5 of ML energy went to inference** and ~2/5 to training.
- **"Train once, serve forever."** This lopsided cost is why inference is its own discipline.

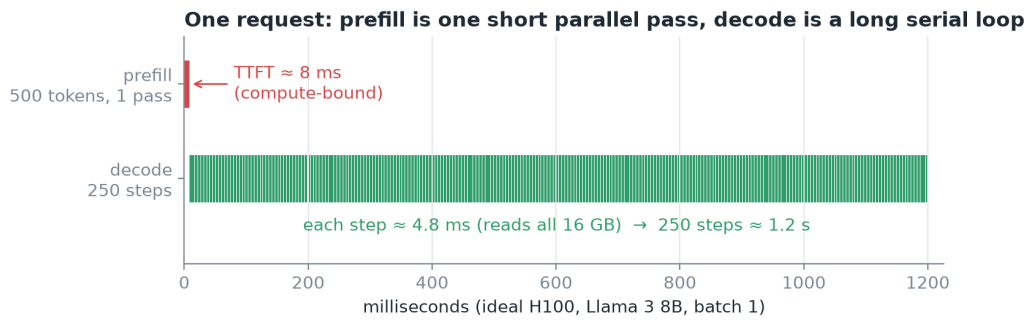
Train once, serve forever



Google ML energy use, 2019 to 2021 (Patterson et al.).

3. One request: prefill and decode

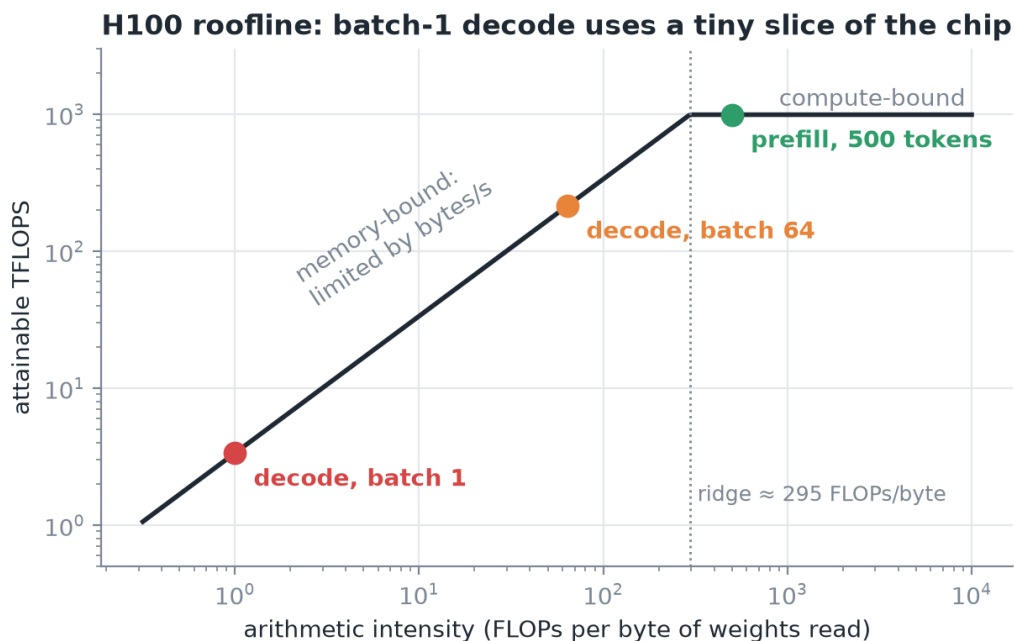
Running example: a chat app serving **Llama 3 8B on one H100**. The prompt is ~500 tokens and the answer is ~250 tokens.



Ideal timings: prefill handles all 500 prompt tokens in one pass (the red sliver is time to first token), then decode runs 250 serial steps of about 4.8 ms each.

- **Prefill:** all 500 prompt tokens go through the model in **one parallel pass**. This causes the pause before the first word, called **TTFT (time to first token)**.
- **Decode:** generate one token, append, run again. **250 sequential steps**.
- **The surprise:** every decode step reads **all 16 GB of weights** from GPU memory (HBM) to produce **one token**.

The bandwidth ceiling (memorize this calculation)



*Batch-1 decode does about 1 FLOP per byte of weights read, far left of the ridge, so memory bandwidth sets its speed.
Prefill sits past the ridge and is compute-bound.*

H100 HBM bandwidth ≈ 3.35 TB/s
 Weights per step = 16 GB
 Time per step = $16 / 3350$ s ≈ 4.8 ms
 Max tokens/s (1 user) $\approx 1 / 4.8$ ms ≈ 200 tok/s

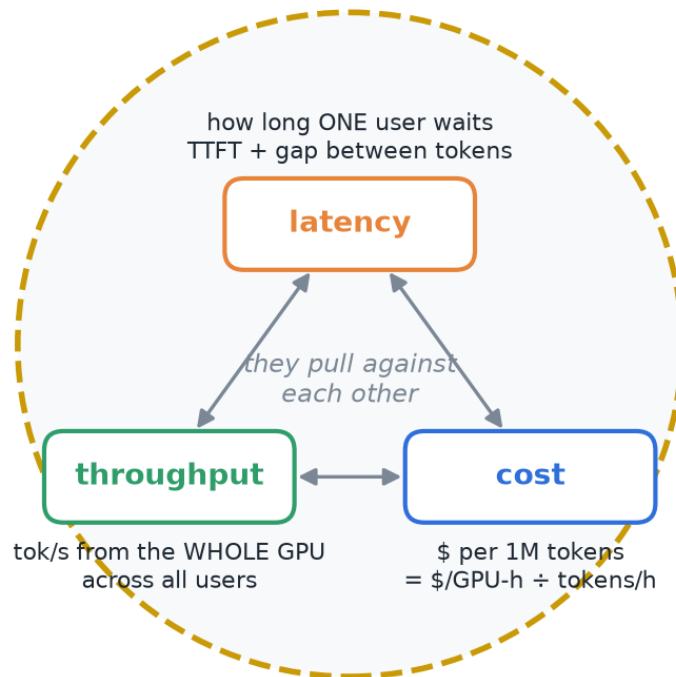
This is a hard ceiling, even with perfect code.

- The chip could do far more math. The limit is **how fast bytes arrive**. Decode at batch 1 is **memory-bandwidth-bound**.

4. The three numbers (and one constraint)

The three numbers an inference engineer watches

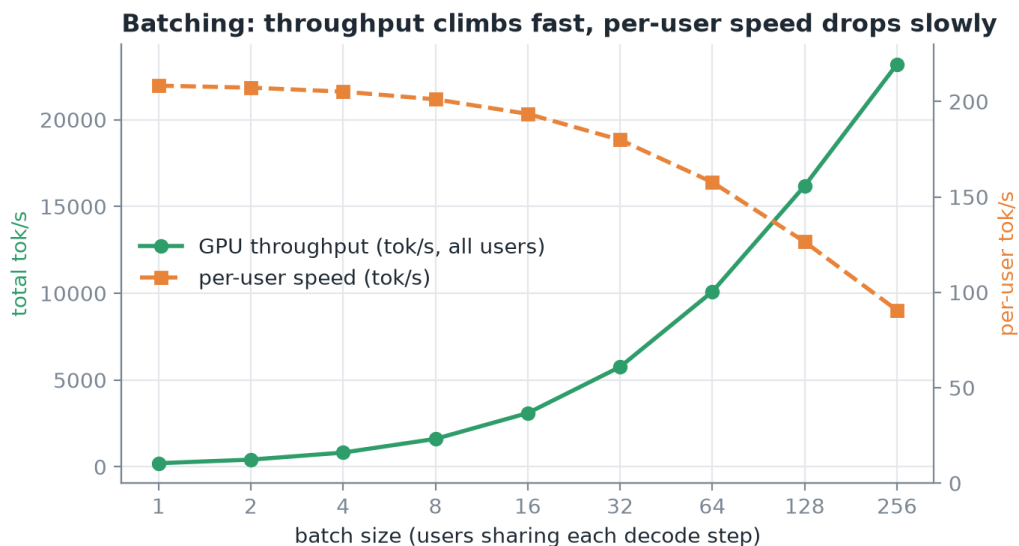
quality: a hard constraint around all three



Quality isn't traded: a faster but worse answer is not a win.

- The gap between tokens is called **ITL** (inter-token latency) or **TPOT** (time per output token).

The first trade-off: batching



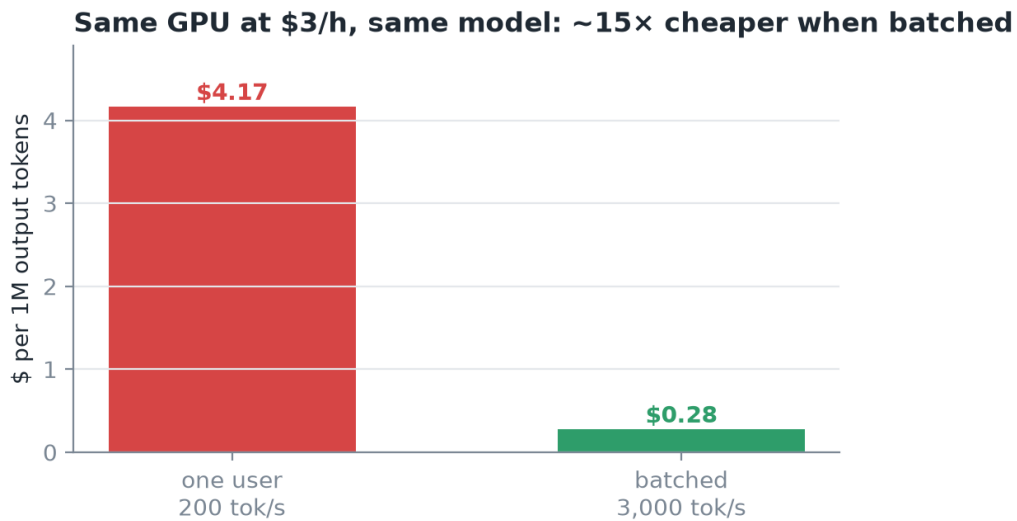
Idealized upper bound: bandwidth model with Llama 3 8B KV cache (128 KB/token). Real engines land lower.

One read of the weights serves every user in the batch, so total throughput climbs steeply while each user's speed falls only gradually.

- **Batch 1:** read 16 GB and get 1 token.

- **Batch 64:** read the same 16 GB and get **64 tokens**, one per user. The weight read is **amortized**.
- Throughput jumps. Each step does more work, so **per-user latency goes up a bit**.
- **"The whole game is a better deal on this trade."** Much of the course is about getting more throughput for less added latency.

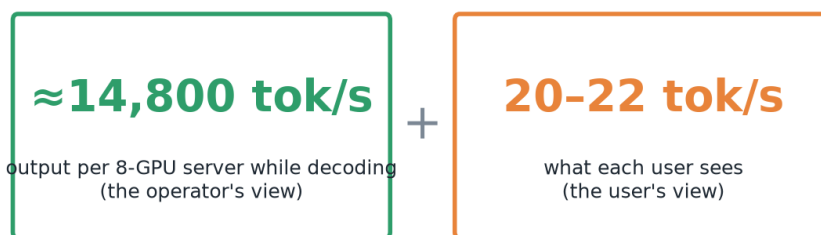
5. Putting money on it



Illustrative prices, real arithmetic. Cost per 1M tokens = $\$3 \div (\text{tok/s} \times 3,600) \times 10^6$.

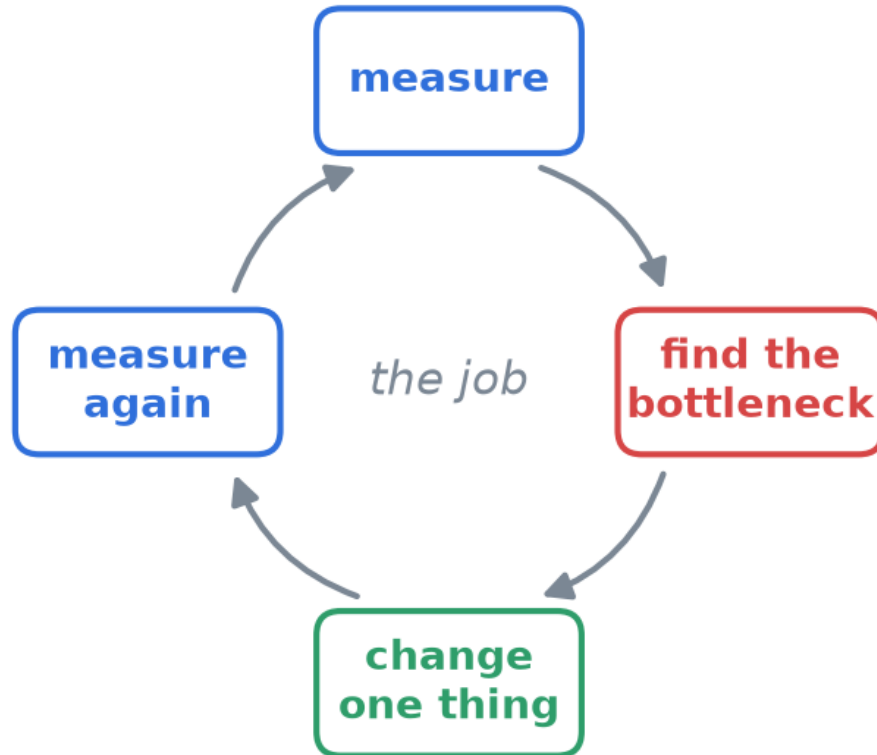
Real-world evidence: - **vLLM (2023)**: up to **24×** the throughput of plain Hugging Face Transformers, with the same GPUs and the same weights.

DeepSeek V3/R1 in production: both numbers good at the same time



6. What an inference engineer does all day

The inference engineer's loop



Six lever areas, from the fleet down to the chip

closer to users

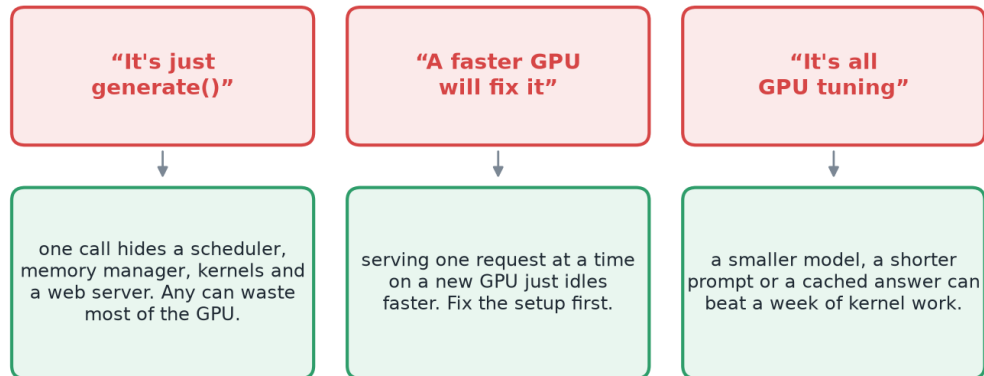
<i>fleet</i>	the fleet	route requests to the right replica; autoscale with traffic
<i>server</i>	scheduling	pick which requests share each step; no stalls
<i>model</i>	precision	8/4-bit weights: fewer bytes moved per step
<i>model</i>	parallelism	split a too-big model across several GPUs
<i>GPU memory</i>	memory	pack the KV cache tightly; reuse shared prompt prefixes
<i>GPU compute</i>	kernels	fuse ops so data makes fewer trips through memory

closer to silicon

Each lever area gets its own module later in the course.

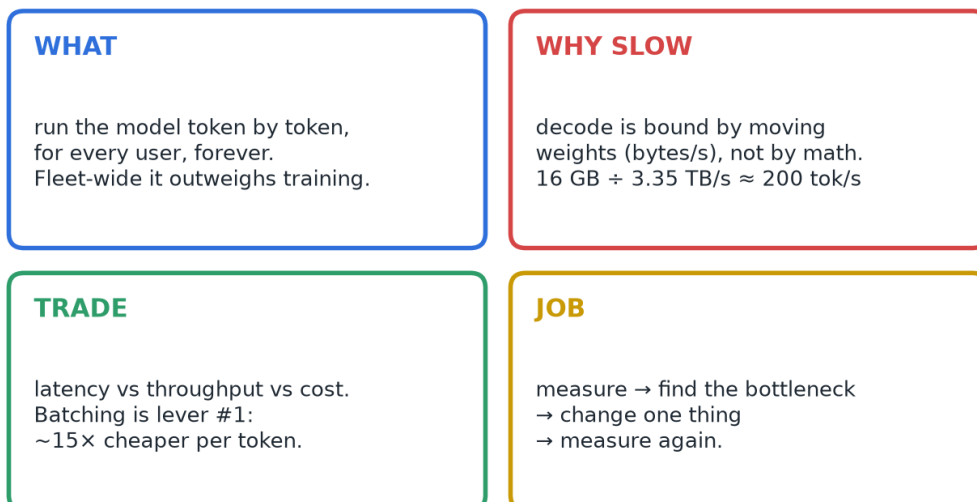
7. Three myths

Three myths (top) and what's actually true (bottom)



8. Recap

Lesson 01 on one card



Interview cheat sheet

Revise from these pages alone. ♦ = my addition beyond the video; everything else is from the lesson.

Say it in 30 seconds

Inference is running a trained model token by token for every request, so fleet-wide it costs more than training. Decode is memory-bandwidth-bound: every step streams all the weights to make one token per sequence. The core trade is latency vs throughput vs cost, and batching is lever one. The job: measure, find the bottleneck, change one thing, repeat.

Only numbers worth memorizing

- **H100**: 3.35 TB/s HBM, ♦ ~989 TFLOPS BF16. **Llama 3 8B**: 16 GB in BF16, ♦ ~128 KB KV per token.
- **Software matters**: batching gives ~15× cheaper tokens; vLLM gets up to 24× naive HF throughput.

Derive, don't memorize

Let P = parameters, B = batch size, BW = memory bandwidth.

1. Decode speed = bandwidth ÷ bytes per step

Every decode step streams all the weights once, so time per step is bytes moved over bytes per second.

$$t_{\text{step}} \approx \frac{2P \text{ bytes}}{BW} = \frac{16 \text{ GB}}{3.35 \text{ TB/s}} \approx 4.8 \text{ ms} \Rightarrow \leq 200 \text{ tok/s per user}$$

Halve the bytes (8-bit weights) and the ceiling doubles. That's why precision is a speed lever.

2. Memory-bound or compute-bound? Compare FLOPs per byte to the ridge ♦

Each weight does a multiply and an add (2 FLOPs) per token and costs 2 bytes to read. One read is shared by all B sequences:

$$\frac{\text{FLOPs}}{\text{byte}} \approx \frac{2P \cdot B}{2P} = B \quad \text{vs} \quad \text{ridge} = \frac{989 \text{ TFLOPS}}{3.35 \text{ TB/s}} \approx 300$$

Batch 1 gives ~1, far below 300, so decode is memory-bound and batching is nearly free speed. Prefill puts ~500 prompt tokens through one read, above the ridge, so it's compute-bound:

$$t_{\text{prefill}} \approx \frac{N_{\text{prompt}} \cdot 2P}{\text{peak FLOPS}} = \frac{500 \times 16 \text{ GFLOP}}{989 \text{ TFLOPS}} \approx 8 \text{ ms}$$

3. Cost = what you pay per hour ÷ what you make per hour

$$\frac{\$}{1\text{M tokens}} = \frac{\$ \text{ per GPU hour}}{\text{tok/s} \times 3600} \times 10^6 \quad \frac{3}{200 \times 3600} \times 10^6 \approx 4.17 \quad \frac{3}{3000 \times 3600} \times 10^6 \approx 0.28$$

4. Why batching isn't free: every sequence brings its own KV cache ♦

Each layer stores one K and one V vector per KV head for every past token:

$$\text{KV/token} = 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times \text{bytes} = 2 \times 32 \times 8 \times 128 \times 2 = 128 \text{ KB}$$

A step reads the weights once plus every sequence's cache (context length C), so steps slow down as the batch grows:

$$t_{\text{step}}(B) \approx \frac{2P + B \cdot C \cdot \text{KV/token}}{\text{BW}}$$

Prefill vs decode

	Prefill	Decode
Work	Whole prompt in one parallel pass	One token per sequence per step, serial
Bottleneck	Compute (FLOPs)	Memory bandwidth (bytes)
User sees it as	TTFT	Gap between tokens (ITL / TPOT)
Grows with	Prompt length	Output length, and batch $\times$ context (KV reads)

Symptom $\rightarrow$ lever (for "how would you fix it?" questions)

Symptom	Likely cause	First levers
High \$/token, GPU underused	Batch too small	Scheduling: batch more requests per step
Batch capped by out-of-memory	KV cache fills memory	Memory: pack KV tightly, reuse shared prefixes
Slow tokens even at low load	Too many weight bytes per step	Precision: 8/4-bit weights
High TTFT	Long prompts or queueing	Shorter prompts, prefix reuse, more replicas
Model doesn't fit one GPU	Size	Parallelism across GPUs, or precision

Rapid-fire Q&A

Question	Crisp answer
Why does batching help, and what does it cost?	One weight read serves all B sequences. The cost is KV memory and slower steps (derivation 4).
◆ Max tok/s for 70B BF16 on 8xH100?	Derivation 1: $140 \text{ GB} \div (8 \times 3.35 \text{ TB/s}) \approx 5.2 \text{ ms/step}$, so $\leq \sim 190 \text{ tok/s}$ before communication overhead.
Cost too high: where do you start?	Measure (utilization, batch size, TTFT/ITL) before changing anything; then check off-GPU wins such as a smaller model, shorter prompts, caching.

Traps to avoid

- "Just call `generate()`" or "buy a faster GPU": both ignore the scheduler, memory and setup.
- Trading away quality: a faster but worse answer is not a win.